

Home > Courses > Syllabus > Session 1

> Session 1 assignment support: Ship Your First AI Endpoint

⌚ Async

Session 1 assignment support: Ship Your First AI Endpoint

1. Start here: what this assignment is

This guide has two paths. **Path A is the basic submission: enough to pass.** Do that first. **Path B is add-ons:** choose one or two only if you have time and energy.

By the end of the basic path, you will have a live `/ask` endpoint on the internet that returns structured JSON with an answer, token usage, and cost — plus a **Streamlit UI** that calls it. That is already a real deployed AI service with a demo front end.

TIP

Course code repo

All starters live in [AI-Internship on GitHub](#). This week: [Week 1 /ask demo \(3 stages\)](#).

Questions? Chat with us

CRITICAL

The Golden Rule (read this first)

Whenever anything goes wrong, copy the entire error message and paste it into Claude Code or Cursor with: "I got this error, please fix it and explain what happened in simple terms."

- That habit fixes most problems.
- You are never stuck. You just haven't pasted the error yet.

Basic submission: what counts

BASIC SUBMISSION: ENOUGH TO PASS

- ! Fork or clone the Week 1 starter repo.
- ! Run it locally with your OpenAI key in `.env` and confirm `.env` is ignored.
- ! Deploy it to Render with `OPENAI_API_KEY` set in Render environment variables.
- ! Prove the live `/ask` endpoint returns JSON with `answer` , `tokens_used` , and `cost_usd` .
- ! Ship a Streamlit UI that calls `/ask` (use the starter demo UI or your own) and screenshot it working.
- ! Submit your live URL on the assignment page. Do not post it publicly.

2. Before you build: what you are building

- Someone sends your service a question, for example: "What is RAG in one sentence?"
- Your service asks an AI model for the answer.
- It sends back a clean, predictable response with `answer` , `tokens_used` , and `cost_usd` .
- Done means a **public HTTPS URL** someone else can hit, not just localhost.

Preflight: accounts and safety

GitHub account

github.com: where your code lives and where Render will fetch from.

OpenAI API key

platform.openai.com → API Keys. Add a small amount of credit (about \$5 is plenty to start). Your key starts with `sk- ...`.

Render account

render.com: hosts your service on the internet. Signup is easiest with GitHub.

Claude Code or Cursor

Your coding partner. Install **one** and keep it working before you start making changes.

CRITICAL

Protect your wallet before you write a single line

Treat your API key like a bank card number. Never post it publicly, never save it in code files, and never upload it to GitHub by mistake.

- Turn OFF auto-recharge: platform.openai.com → Settings → Billing → Auto recharge = OFF.
- Set a low monthly budget + alert: platform.openai.com → Settings → Limits/Budgets. Budget \$5-\$10, email alert at \$3.
- If you think your key leaked: revoke the old key immediately, create a new one, and update your `.env` + Render environment variables.

3. Path A: Basic submission

Do these steps first. When the basic checklist is done, you have passed the assignment. Stop there if you are tired.

Step 1: Get the starter code



2. Click **Fork** (top right on GitHub) to create your own copy.
3. On your fork, click the green **Code** button and copy the fork URL.

PROMPT 1: CLONE THE REPO

[Copy](#)

I am a complete beginner. Please clone this GitHub repository to my computer and open it: [PASTE YOUR FORK URL HERE].
Then give me a one-paragraph plain-English summary of what this project does and what the 5 stages are.

Read the AI's summary. You now understand the project you're deploying.

Step 2: Set up your environment

When you get the setup prompt, create your venv, install dependencies, and edit your `.env` file locally. Paste your OpenAI key only where it belongs.

PROMPT 2: SET UP THE PROJECT

[Copy](#)

Set this project up for me step by step:

1. Create a Python virtual environment and activate it
2. Install everything in requirements.txt
3. Copy `.env.example` to `.env` and tell me exactly where to paste my

OpenAI API key

4. Do NOT print my API key back to me at any point

Explain each step in one simple sentence as you go. I have never done this before.

WATCH OUT

Key hygiene reminder

- Paste your key only after `OPENAI_API_KEY=` in your local `.env` file.
- Never paste the key into chat.
- Never commit `.env` to GitHub. Only commit `.env.example`.

Step 3: Run locally and test once

PROMPT 3: RUN AND SMOKE TEST

 Copy

Now:

1. Start the stage 5 server with uvicorn on port 8000
2. Run `python test_all_stages.py` and show me the results
3. Send a test question to the `/ask` endpoint using `curl` and show me the JSON response
4. Confirm the response contains three things: `answer`, `tokens_used`, and `cost_usd`

If anything fails, fix it and explain what was wrong in simple terms.

READY WHEN

What success looks like

- You see a JSON response with curly brackets.
- It includes `answer`, a numeric `tokens_used`, and a numeric `cost_usd`.
- Take a screenshot for your LinkedIn post.

Step 4: Push your code back to GitHub

PROMPT 4: PUSH TO GITHUB (AND VERIFY `.ENV` IS NOT UPLOADED)

 Copy

Before doing anything else, run `git status` and confirm that `.env` does NOT appear in the list of files to be committed, and that `.env` is listed inside the `.gitignore` file.

If `.env` is not in `.gitignore`, add it first.

Only then commit all my changes with a sensible commit message and push them to my fork on GitHub.

After pushing, confirm once more that `.env` was not uploaded.

Double-check with your own eyes: after pushing, open my repo on `github.com` in the browser and look at the file list. You should see `.env.example` but you should NOT see `.env`.

If you can see `.env` on GitHub, your key has leaked. Revoke the key at `platform.openai.com`, create a new one, update `.env` locally and on Render, and ask me to remove the old secret from the repo and its history immediately.`

Step 5: Deploy to Render (in your browser)

This part happens in your web browser, not inside Claude Code or Cursor.

1. Go to `render.com` and log in.
2. Click **New +** → **Web Service**.
3. Connect your GitHub account (if asked), then select your forked repo.
4. Fill in the settings: - Runtime: Python - Build Command: `pip install -r requirements.txt` - Start Command: `uvicorn main:app --host 0.0.0.0 --port $PORT`
5. Add Environment Variables: - Key: `OPENAI_API_KEY` - Value: your key (paste it here in Render; this is the safe place for it).
6. Click **Create Web Service** and wait for it to turn green ("Live").

7. Copy your public URL (looks like `https://your-service.onrender.com`).

Step 6: Prove it works from the outside

PROMPT 5: TEST THE LIVE URL

[Copy](#)

My service is now live at `https://YOUR-SERVICE.onrender.com`.
Run this exact curl command and show me the full response.

Confirm the response is valid JSON with `answer`, `tokens_used`, and `cost_usd`.

CURL AGAINST THE LIVE URL (REPLACE YOUR-SERVICE)

[Copy](#)

```
curl -s -X POST https://YOUR-SERVICE.onrender.com/ask \  
  -H "Content-Type: application/json" \  
  -d '{"question": "What is RAG in one sentence?", "model": "gpt-4o-mini"}'
```

Screenshot the curl request and the JSON response. This is your headline proof.

Step 7: Run the Streamlit UI

The Week 1 starter includes a Streamlit demo. Run it so it calls your `/ask` API (local or live). A curl-only submission without a UI does not pass.

PROMPT 6: STREAMLIT UI AGAINST /ASK

[Copy](#)

This repo includes a Streamlit demo UI. Please:

1. Show me how to run it locally (exact command)
2. Point it at my `/ask` API (local `http://127.0.0.1:8000` or my live

Render URL)

3. Confirm I can type a question in the UI and get back answer, tokens_used, and cost_usd
4. Tell me what screenshot to attach as proof

If the Streamlit app is missing or broken, create a minimal one-page Streamlit UI that POSTs to /ask and displays the JSON fields.

4. Stop point: basic submission is done

READY WHEN

Stop here if you need to

If your live URL returns structured JSON with `answer`, `tokens_used`, and `cost_usd`, and you have a working Streamlit UI that calls `/ask`, the basic submission is done. The add-ons below are useful, but optional.

BEFORE YOU MOVE TO ADD-ONS, CONFIRM THIS

- ! The service runs locally.
- ! The service is deployed on Render or equivalent.
- ! The live `/ask` URL returns JSON with `answer`, `tokens_used`, and `cost_usd`.
- ! Streamlit UI runs and successfully calls `/ask` (screenshot ready).
- ! Your `.env` file is not on GitHub.
- ! You have a curl command and response ready to share privately.

5. Path B: Optional add-ons

Add-on 1: Prove the guardrail works **Recommended**

Trigger `force_bad` , screenshot validation catching malformed output, and explain what would have happened without the guardrail.

Add-on 2: Explain model choice and cost

Write 1-2 sentences on the default model, why it fits quality/cost/latency, and approximate cost per call from your real `cost_usd` .

Add-on 3: Polish the README

Add setup, local run, deploy, and curl instructions. Keep secrets out. Include `.env.example` , never `.env` .

Add-on 4: Try extra smoke tests

Ask 3 different questions and compare answers, token usage, and cost behavior.

Add-on 5: LinkedIn proof post

Post screenshots or a short screen recording only. Credit the programme. Never include the live URL.

Add-on prompt: prove the guardrail works

The repo includes a built-in switch called `force_bad` to make the model produce deliberately malformed output.

ADD-ON PROMPT: TRIGGER THE GUARDRAIL Copy

This repo has a `force_bad` option (stage 3 and above) that makes the model produce deliberately malformed output. Please:

1. Show me how to trigger it

2. Trigger it and show me what the guardrail does when validation fails
3. Explain in two sentences what would have happened WITHOUT the guardrail

Then turn `force_bad` off again.

Add-on prompt: explain your choices

ADD-ON PROMPT: EXPLAIN MY CHOICES (SHORT ANSWERS)

 Copy

Based on this project, help me write three short answers in my own plain words:

1. Which model this uses by default and a one-sentence justification (quality vs cost vs latency)
2. The approximate cost per call, using the real `cost_usd` from my live response
3. What the guardrail does when validation fails (retry, safe default, or controlled error), based on what the code actually does

Keep each answer to one or two sentences. No jargon.

6. Share + troubleshoot

CRITICAL

Do NOT share your live URL publicly

Never post your Render (or other) service URL on LinkedIn, Twitter/X, blogs, or any public page. Anyone with the link can call your `/ask` endpoint and burn through your OpenAI credits. Keep the live URL private to the cohort channels below.

- LinkedIn: screenshots / screen recording only. Never paste the live URL.
- Public repos / READMEs: do not put the production URL in the README if the repo is public.

- Cohort only: Maven submission channels are the right place for feedback from classmates and instructors.

- In Maven: post your live URL, the curl command, a Streamlit screenshot, and one specific thing you want feedback on.
- On LinkedIn: attach 2 to 4 screenshots (curl request, JSON response with answer + tokens + cost, Streamlit UI, optional guardrail demo). Credit the program. Do **not** include your live URL.

BASIC SUBMISSION CHECKLIST

- ! OpenAI auto-recharge is OFF.
- ! Your OpenAI key is stored in `.env` and `.env` is ignored (never committed).
- ! Built from the Week 1 starter repo and runs locally.
- ! `test_all_stages.py` passes.
- ! Deployed to Render with a public HTTPS URL.
- ! curl against the live URL returns valid structured JSON.
- ! The response includes `tokens_used` and `cost_usd`.
- ! Streamlit UI calls `/ask` and you have a screenshot of it working.
- ! You submitted the live URL + curl command + Streamlit screenshot on the assignment page.

OPTIONAL ADD-ONS

- You triggered `force_bad` and saw the guardrail catch malformed output.
- You can state per-call cost and defend your model choice in one sentence.
- You polished the README with setup/deploy instructions.
- You published a LinkedIn post with screenshots only (no live URL, no API key).

Troubleshooting (6 common problems)

- **Command not found: python / pip:** paste the error into your coding tool and ask it to detect your OS and install Python 3.11+ + venv.
- **Invalid API key / 401:** your key is missing or wrong. Check Render environment variables and your local `.env` (no spaces, no quotes).
- **Insufficient quota / 429:** you need more credit. Go to [platform.openai.com](https://platform.openai.com/billing) → Billing and add ~\$5 manually (keep auto-recharge off).
- **You think the key leaked:** revoke immediately, create a new key, update `.env` locally and Render, and remove the secret from GitHub history if needed.
- **Render deploy fails:** copy the Render log error, paste it into your AI tool, and ask what you need to change.
- **Live URL times out on first try:** free Render services can sleep. Wait 60 seconds and retry.

[← Back to Session 1: Foundations - the reliable reasoning component](#) · [Full syllabus](#)

PREVIOUS

[← Module B - Mastering AI Engineering: the free-resources...](#)



Training the AI-Native Workforce. Custom programs that deliver measurable productivity outcomes in weeks.

Operated by **Snapdrum**

PRODUCTS

LEARN

TAI for Teams

Tool Guides

Seats for teams

Live Programs

Pricing

RESOURCES

Case Studies

ROI Calculator

Team AI Playbook

COMPANY

About

Courses

Contact

© 2026 TAI Labs

Terms Privacy Security